

Python Mastery: Netmiko

Multi-vendor Network Automation via SSH

1. Introduction to Netmiko

Netmiko is a high-level Python library built on top of Paramiko. It simplifies SSH connections to network devices (routers, switches, firewalls) by abstracting vendor-specific CLI behaviors like command prompts and paging.

Why Netmiko?

- **Vendor Support:** Supports 100+ platforms (Cisco, Juniper, Arista, HP, etc.).
- **Prompt Management:** Automatically handles "Press any key to continue" or "yes/no" prompts.
- **Configuration Mode:** Switches to config mode automatically when sending config commands.

2. Connecting to a Device

The `ConnectHandler` is the entry point for Netmiko. You define the device type, IP, and credentials.

```
from netmiko import ConnectHandler

cisco_device = {
    'device_type': 'cisco_ios',
    'host': '10.0.0.1',
    'username': 'admin',
    'password': 'cisco123',
    'secret': 'enable_pass',
}

net_connect = ConnectHandler(**cisco_device)
net_connect.enable() # Enter privileged EXEC mode
```

3. Sending Show Commands

Use `send_command()` to retrieve operational data. Netmiko handles waiting for the prompt to return.

```
output = net_connect.send_command('show ip int brief')
print(output)

# Get structured data using TextFSM (optional)
output_structured = net_connect.send_command('show ip int brief', use_textfsm=True)
```

4. Sending Configuration Commands

Use `send_config_set()` to apply multiple configuration lines. It automatically enters and exits configuration mode.

```
config_commands = [
    'interface Loopback0',
    'description Automated via Netmiko',
    'ip address 1.1.1.1 255.255.255.255'
]

output = net_connect.send_config_set(config_commands)
print(output)
net_connect.save_config() # copy run start
```

5. Error Handling & Disconnecting

Always close the connection to release resources on the network device.

```
net_connect.disconnect()
```